Challenge 2

AI-Powered Knowledge Base System

Ideation

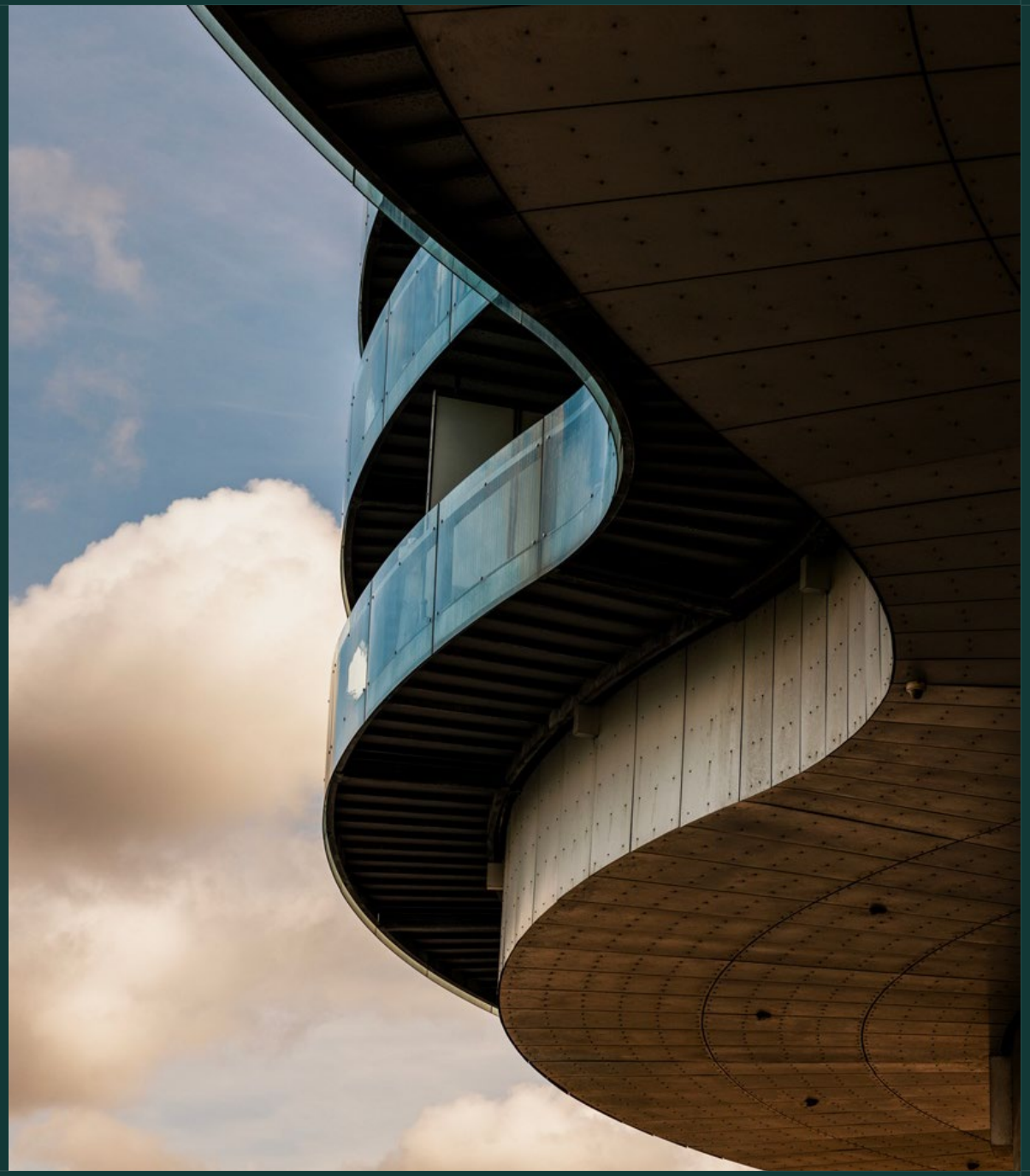
Design and implement an **intelligent Knowledge Base solution** powered by GenAI.

The goal is to go beyond a simple Q&A bot - build a system that understands, organizes, and retrieves knowledge intelligently.

Your solution should leverage AI to:

- Ingest and structure knowledge from documents, APIs, or provided data sources.
- Allow natural language querying with relevant, explainable responses.
- Learn from user interactions, refining relevance or summaries over time.
- Include semantic search or retrieval-augmented generation (RAG) for grounded answers.
- GenAI should act as the reasoning and retrieval engine, not just a text generator - use it to connect, interpret, and synthesize knowledge meaningfully.

To make the problem tangible and show business value, each team should select a specific domain and tailor the knowledge base to that context.



Expected Outcomes

- **Working Prototype:**
  A functioning system that allows users to add, search, and query information using natural language.
- **Knowledge Artifacts:**
  The system should demonstrate structured output - e.g., knowledge graphs, entity relationships, embeddings, or summarized content snippets.
- **Live Demo:**
  Present your solution end-to-end - from ingestion and indexing to natural language answers and reasoning results.
- **Approach & Rationale:**
  Explain your system's architecture, how GenAI is used (indexing, semantic search, summarization, etc.), and **why your design choices matter** (e.g., scalability, modularity, novelty).

Tech Stack

Core Technologies

- Runtime Environment: Dockerized application running on a Linux base image.
- Backend: Indicative choices: Python (FastAPI, Flask) or Node.js (Express, NestJS).
- Frontend (optional): Indicative choices: React, Vue.js, or a simple HTML/JS UI for querying and visualization.
- Database/Search Layer: PostgreSQL, SQLite, or vector databases (e.g., FAISS, Chroma, Weaviate, or Elasticsearch).
- AI / LLM Frameworks: OpenAI API, Hugging Face Transformers, LangChain, or LlamaIndex for retrieval and reasoning.

Supporting Tools

- Containerization: Docker / Docker Compose for multi-service setups.
- Embedding / RAG Pipelines: Use of bm25, TF-IDF, or dense embeddings for semantic retrieval.
- Documentation: Clear README and optional architectural diagram (Mermaid, draw.io, or similar).